基于Flutter的安卓开发学习移动应用软件源代码

1 封面信息

- 软件名称：基于Flutter的安卓开发学习移动应用软件
- 版本号：V1.0.0
- 应用标题：Flutter组件实验手册
- Android 包名：`com.example.flutter_dudo`
- 代码规模：约 2009 行自研代码

2 源代码属性结构说明

- 技术栈：Flutter、Dart、Kotlin、Android Gradle Kotlin DSL
- 源码根目录：`D:\workspace\front\flutter_dudo`
- 一级目录划分：`android/` 用于 Android 构建与权限声明，`lib/` 用于 Flutter 页面与组件实现，`docs/` 用于软著材料输出。
- 二级目录划分：`lib/pages/` 承载页面模块，`lib/widgets/` 承载通用组件，`lib/pages/hardware/` 承载设备能力示例。
- 三级目录划分：`lib/pages/navigation/` 存放导航结果模型与详情页，`android/app/src/main/` 存放 Android 主配置。
- 选入范围说明：纳入应用配置、Flutter 页面、通用组件、Android 权限清单和启动入口，不纳入第三方依赖、构建缓存和生成产物。

3 三级目录结构树

```
├── pubspec.yaml
├── android
│   └── app
│       ├── build.gradle.kts
│       └── src
└── lib
    ├── main.dart
    ├── app.dart
    ├── pages
    │   ├── home_page.dart
    │   ├── text_display_page.dart
    │   ├── layout_widgets_page.dart
    │   ├── button_interaction_page.dart
    │   ├── form_input_page.dart
    │   ├── feedback_page.dart
    │   ├── navigation_page.dart
    │   ├── navigation
    │   ├── state_lifting_page.dart
    │   ├── theme_page.dart
    │   ├── async_list_page.dart
    │   ├── hardware_hub_page.dart
    │   └── hardware
    └── widgets
        ├── demo_page_template.dart
        ├── section_card.dart
        ├── code_block.dart
        └── network_image_with_fallback.dart
```

4 按目录顺序整理的源代码

4.1 pubspec.yaml

文件路径：`pubspec.yaml`

文件作用：定义应用名称、版本号、Flutter SDK 约束和硬件插件依赖。

所属模块：项目配置

```
name: flutter_dudo
description: "Flutter 常用组件实验手册（初学者学习 Demo）"
# The following line prevents the package from being accidentally published to
# pub.dev using `flutter pub publish`. This is preferred for private packages.
publish_to: 'none' # Remove this line if you wish to publish to pub.dev

# The following defines the version and build number for your application.
# A version number is three numbers separated by dots, like 1.2.43
# followed by an optional build number separated by a +.
# Both the version and the builder number may be overridden in flutter
# build by specifying --build-name and --build-number, respectively.
# In Android, build-name is used as versionName while build-number used as versionCode.
# Read more about Android versioning at https://developer.android.com/studio/publish/versioning
# In iOS, build-name is used as CFBundleShortVersionString while build-number is used as CFBundleVersion.
# Read more about iOS versioning at
# https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/CoreFoundationKeys.html
# In Windows, build-name is used as the major, minor, and patch parts
# of the product and file versions while build-number is used as the build suffix.
version: 1.0.0+1

environment:
  sdk: ^3.11.4

# Dependencies specify other packages that your package needs in order to work.
# To automatically upgrade your package dependencies to the latest versions
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers below to
# the latest version available on pub.dev. To see which dependencies have newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter
  geolocator: ^13.0.2
  sensors_plus: ^6.1.1
  image_picker: ^1.1.2
  file_picker: ^8.1.2
  flutter_blue_plus: ^1.35.5

# For information on the generic Dart part of this file, see the
# following page: https://dart.dev/tools/pub/pubspec

# The following section is specific to Flutter packages.
flutter:

  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
  uses-material-design: true

  # To add assets to your application, add an assets section, like this:
  # assets:
  #   - images/a_dot_burr.jpeg
  #   - images/a_dot_ham.jpeg

  # An image asset can refer to one or more resolution-specific "variants", see
  # https://flutter.dev/to/resolution-aware-images

  # For details regarding adding assets from package dependencies, see
  # https://flutter.dev/to/asset-from-package

  # To add custom fonts to your application, add a fonts section here,
  # in this "flutter" section. Each entry in this list should have a
  # "family" key with the font family name, and a "fonts" key with a
  # list giving the asset and other descriptors for the font. For
  # example:
  # fonts:
  #   - family: Schyler
  #     fonts:
```

```
  #       - asset: fonts/Schyler-Regular.ttf
  #       - asset: fonts/Schyler-Italic.ttf
  #         style: italic
  #   - family: Trajan Pro
  #     fonts:
  #       - asset: fonts/TrajanPro.ttf
  #       - asset: fonts/TrajanPro_Bold.ttf
  #         weight: 700
  #
  # For details regarding fonts from package dependencies,
  # see https://flutter.dev/to/font-from-package
```

4.2 android/app/build.gradle.kts

文件路径：`android/app/build.gradle.kts`

文件作用：定义 Android 应用命名空间、应用包名、版本号和构建类型。

所属模块：Android 构建配置

```
plugins {
    id("com.android.application")
    id("kotlin-android")
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
    id("dev.flutter.flutter-gradle-plugin")
}

android {
    namespace = "com.example.flutter_dudo"
    compileSdk = flutter.compileSdkVersion
    ndkVersion = flutter.ndkVersion

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = JavaVersion.VERSION_17.toString()
    }

    defaultConfig {
        // TODO: Specify your own unique Application ID (https://developer.android.com/studio/build/application-id.html).
        applicationId = "com.example.flutter_dudo"
        // You can update the following values to match your application needs.
        // For more information, see: https://flutter.dev/to/review-gradle-config.
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }

    buildTypes {
        release {
            // TODO: Add your own signing config for the release build.
            // Signing with the debug keys for now, so `flutter run --release` works.
            signingConfig = signingConfigs.getByName("debug")
        }
    }
}

flutter {
    source = "../.."
}
```

4.3 android/app/src/main/AndroidManifest.xml

文件路径：`android/app/src/main/AndroidManifest.xml`

文件作用：声明定位、相机、蓝牙和媒体读取权限，并配置主入口 Activity。

所属模块：Android 权限配置

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
    <uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
    <application
        android:label="flutter_dudo"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTop"
            android:taskAffinity=""
            android:theme="@style/LaunchTheme"
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layoutDirection|fontScale|screenLayout|de
            android:hardwareAccelerated="true"
            android:windowSoftInputMode="adjustResize">
            <!-- Specifies an Android theme to apply to this Activity as soon as
                 the Android process has started. This theme is visible to the user
                 while the Flutter UI initializes. After that, this theme continues
                 to determine the Window background behind the Flutter UI. -->
            <meta-data
              android:name="io.flutter.embedding.android.NormalTheme"
              android:resource="@style/NormalTheme"
              />
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <!-- Don't delete the meta-data below.
             This is used by the Flutter tool to generate GeneratedPluginRegistrant.java -->
        <meta-data
            android:name="flutterEmbedding"
            android:value="2" />
    </application>
    <!-- Required to query activities that can process text, see:
         https://developer.android.com/training/package-visibility and
         https://developer.android.com/reference/android/content/Intent#ACTION_PROCESS_TEXT.

         In particular, this is used by the Flutter engine in io.flutter.plugin.text.ProcessTextPlugin. -->
    <queries>
        <intent>
            <action android:name="android.intent.action.PROCESS_TEXT"/>
            <data android:mimeType="text/plain"/>
        </intent>
    </queries>
</manifest>
```

4.4 android/app/src/main/kotlin/com/example/flutter_dudo/MainActivity.kt

文件路径：`android/app/src/main/kotlin/com/example/flutter_dudo/MainActivity.kt`

文件作用：提供 FlutterActivity 宿主，作为 Android 端启动壳。

所属模块：Android 启动入口

```
package com.example.flutter_dudo

import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity()
```

4.5 lib/main.dart

文件路径：`lib/main.dart`

文件作用：完成 Flutter 绑定初始化并启动根组件。

所属模块：应用入口

```
import 'package:flutter/widgets.dart';
import 'app.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const FlutterWidgetLabApp());
}
```

4.6 lib/app.dart

文件路径：`lib/app.dart`

文件作用：配置 MaterialApp、主题模式、命名路由和首页挂载。

所属模块：根应用

```
import 'package:flutter/material.dart';
import 'pages/home_page.dart';
import 'pages/navigation/route_detail_page.dart';

class FlutterWidgetLabApp extends StatefulWidget {
  const FlutterWidgetLabApp({super.key});

  @override
  State<FlutterWidgetLabApp> createState() => _FlutterWidgetLabAppState();
}

class _FlutterWidgetLabAppState extends State<FlutterWidgetLabApp> {
  ThemeMode _themeMode = ThemeMode.light;

  void _updateThemeMode(ThemeMode mode) {
    setState(() {
      _themeMode = mode;
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter 组件实验手册',
      debugShowCheckedModeBanner: false,
      themeMode: _themeMode,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        appBarTheme: const AppBarTheme(centerTitle: true),
      ),
      darkTheme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.blue,
          brightness: Brightness.dark,
        ),
        appBarTheme: const AppBarTheme(centerTitle: true),
      ),
      routes: <String, WidgetBuilder>{
        RouteDetailPage.routeName: (BuildContext context) {
          final Object? args = ModalRoute.of(context)?.settings.arguments;
          final String message = args is String ? args : '未传递参数';
          return RouteDetailPage(message: message);
        },
      },
      home: HomePage(
        currentThemeMode: _themeMode,
        onThemeModeChanged: _updateThemeMode,
      ),
    );
  }
}
```

4.7 lib/pages/home_page.dart

文件路径：`lib/pages/home_page.dart`

文件作用：展示全部学习模块列表并负责进入各功能页面。

所属模块：首页模块

```
import 'package:flutter/material.dart';
import 'async_list_page.dart';
import 'button_interaction_page.dart';
import 'feedback_page.dart';
import 'form_input_page.dart';
import 'hardware_hub_page.dart';
import 'layout_widgets_page.dart';
import 'navigation_page.dart';
import 'state_lifting_page.dart';
import 'text_display_page.dart';
import 'theme_page.dart';

class HomePage extends StatelessWidget {
  const HomePage({
    super.key,
    required this.currentThemeMode,
    required this.onThemeModeChanged,
  });

  final ThemeMode currentThemeMode;
  final ValueChanged<ThemeMode> onThemeModeChanged;

  @override
  Widget build(BuildContext context) {
    final List<_ModuleItem> modules = <_ModuleItem>[
      _ModuleItem(
        title: '文本与显示组件学习',
        subtitle: 'Text / RichText / Image / Icon',
        icon: Icons.text_fields,
        builder: (BuildContext context) => const TextDisplayPage(),
      ),
      _ModuleItem(
        title: '常见布局组件学习',
        subtitle: 'Row / Column / Expanded / Wrap / Stack',
        icon: Icons.dashboard_customize_outlined,
        builder: (BuildContext context) => const LayoutWidgetsPage(),
      ),
      _ModuleItem(
        title: '按钮与点击交互学习',
        subtitle: 'ElevatedButton / OutlinedButton / IconButton',
        icon: Icons.touch_app_outlined,
        builder: (BuildContext context) => const ButtonInteractionPage(),
      ),
      _ModuleItem(
        title: '输入组件与表单学习',
        subtitle: 'TextField / Form / Dropdown / Switch',
        icon: Icons.edit_note_outlined,
        builder: (BuildContext context) => const FormInputPage(),
      ),
      _ModuleItem(
        title: '弹窗与提示反馈学习',
        subtitle: 'SnackBar / Dialog / BottomSheet',
        icon: Icons.notifications_active_outlined,
        builder: (BuildContext context) => const FeedbackPage(),
      ),
      _ModuleItem(
        title: '导航与页面切换学习',
        subtitle: 'Navigator.push / pushNamed / pop',
        icon: Icons.route_outlined,
        builder: (BuildContext context) => const NavigationPage(),
      ),
      _ModuleItem(
        title: '状态更新与父子传值学习',
        subtitle: 'setState + 回调函数传值',
        icon: Icons.swap_horiz_outlined,
        builder: (BuildContext context) => const StateLiftingPage(),
      ),
      _ModuleItem(
```

```
        title: '主题样式学习',
        subtitle: 'ThemeData / ThemeMode',
        icon: Icons.palette_outlined,
        builder: (BuildContext context) => ThemeStudyPage(
          currentMode: currentThemeMode,
          onThemeModeChanged: onThemeModeChanged,
        ),
      ),
      _ModuleItem(
        title: '异步加载与列表展示学习',
        subtitle: 'FutureBuilder + ListView',
        icon: Icons.cloud_download_outlined,
        builder: (BuildContext context) => const AsyncListPage(),
      ),
      _ModuleItem(
        title: '硬件设备示例学习',
        subtitle: 'GPS / 传感器 / 相机 / 文件 / 蓝牙',
        icon: Icons.memory_outlined,
        builder: (BuildContext context) => const HardwareHubPage(),
      ),
    ];

    return Scaffold(
      appBar: AppBar(title: const Text('Flutter 组件实验手册')),
      body: ListView.separated(
        padding: const EdgeInsets.all(16),
        itemCount: modules.length,
        separatorBuilder: (BuildContext context, int index) =>
            const SizedBox(height: 10),
        itemBuilder: (BuildContext context, int index) {
          final _ModuleItem item = modules[index];
          return Card(
            child: ListTile(
              leading: CircleAvatar(child: Icon(item.icon)),
              title: Text(item.title),
              subtitle: Text(item.subtitle),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute<Widget>(builder: item.builder),
                );
              },
            ),
          );
        },
      ),
    );
  }
}

class _ModuleItem {
  _ModuleItem({
    required this.title,
    required this.subtitle,
    required this.icon,
    required this.builder,
  });

  final String title;
  final String subtitle;
  final IconData icon;
  final WidgetBuilder builder;
}
```

4.8 lib/pages/text_display_page.dart

文件路径：`lib/pages/text_display_page.dart`

文件作用：演示文本、富文本、图片、图标和开关式参数调整。

所属模块：基础展示模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';
import '../widgets/network_image_with_fallback.dart';
```

```
class TextDisplayPage extends StatefulWidget {
  const TextDisplayPage({super.key});

  @override
  State<TextDisplayPage> createState() => _TextDisplayPageState();
}

class _TextDisplayPageState extends State<TextDisplayPage> {
  double _fontSize = 18;
  bool _ellipsis = true;

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '文本与显示组件',
      scenario: '用于展示标题、正文、标签、头像和图片，是几乎所有页面都会用到的基础能力。',
      tips: const <String>[
        '修改 Text 的 fontSize、fontWeight、color',
        '把 maxLines 从 1 改成 2 或 3，观察省略号变化',
        '替换网络图片 URL，故意写错看看兜底效果',
      ],
      keyCode: '''
Text(
  'Flutter 文本展示示例',
  style: TextStyle(fontSize: fontSize, fontWeight: FontWeight.bold),
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
)

Image.network(
  url,
  errorBuilder: (_, __, ___) => const Icon(Icons.broken_image),
)
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Text(
            'Flutter 文本展示示例（可调字号）',
            maxLines: _ellipsis ? 1 : 2,
            overflow: _ellipsis ? TextOverflow.ellipsis : TextOverflow.visible,
            style: TextStyle(fontSize: _fontSize, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 10),
          RichText(
            text: TextSpan(
              style: Theme.of(context).textTheme.bodyMedium,
              children: const <InlineSpan>[
                TextSpan(text: 'RichText 可以在同一行里设置 '),
                TextSpan(
                  text: '不同颜色',
                  style: TextStyle(color: Colors.blue),
                ),
                TextSpan(text: ' 和 '),
                TextSpan(
                  text: '不同粗细',
                  style: TextStyle(fontWeight: FontWeight.bold),
                ),
                TextSpan(text: '。'),
              ],
            ),
          ),
          const SizedBox(height: 10),
          const Row(
            children: <Widget>[
              Icon(Icons.star, color: Colors.orange),
              SizedBox(width: 8),
              Chip(label: Text('常用展示组件')),
              SizedBox(width: 8),
              CircleAvatar(child: Text('F')),
            ],
          ),
          const SizedBox(height: 14),
          const NetworkImageWithFallback(url: 'https://picsum.photos/600/260'),
          const SizedBox(height: 14),
          Text('字体大小：${_fontSize.toStringAsFixed(0)}'),
          Slider(
            min: 14,
            max: 32,
            value: _fontSize,
```

```
            onChanged: (double value) {
              setState(() {
                _fontSize = value;
              });
            },
          ),
          SwitchListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('开启单行省略'),
            value: _ellipsis,
            onChanged: (bool value) {
              setState(() {
                _ellipsis = value;
              });
            },
          ),
        ],
      ),
    );
  }
}
```

4.9 lib/pages/layout_widgets_page.dart

文件路径：`lib/pages/layout_widgets_page.dart`

文件作用：演示 Row、Expanded、Wrap、Stack 等常见布局组件。

所属模块：布局模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class LayoutWidgetsPage extends StatefulWidget {
  const LayoutWidgetsPage({super.key});

  @override
  State<LayoutWidgetsPage> createState() => _LayoutWidgetsPageState();
}

class _LayoutWidgetsPageState extends State<LayoutWidgetsPage> {
  double _spacing = 8;

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '常见布局组件',
      scenario: '当你需要把组件横向、纵向、换行或层叠摆放时，会频繁使用 Row、Column、Wrap、Stack。',
      tips: const <String>[
        '修改 Expanded 的 flex 比例，观察占位变化',
        '把 Wrap 的 spacing / runSpacing 改大或改小',
        '调整 Stack 中 Positioned 的 top / right 值',
      ],
      keyCode: '''
Row(
  children: const [
    Expanded(flex: 1, child: DemoBox('A')),
    SizedBox(width: 8),
    Expanded(flex: 2, child: DemoBox('B')),
  ],
)
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          const Text('1) Row + Expanded'),
          const SizedBox(height: 8),
          Row(
            children: <Widget>[
              const Expanded(flex: 1, child: _DemoColorBox(label: 'flex:1')),
              SizedBox(width: _spacing),
              const Expanded(flex: 2, child: _DemoColorBox(label: 'flex:2')),
            ],
          ),
          const SizedBox(height: 16),
          const Text('2) Wrap 自动换行'),
```

```
        const SizedBox(height: 8),
        Wrap(
          spacing: _spacing,
          runSpacing: _spacing,
          children: const <Widget>[
            Chip(label: Text('Flutter')),
            Chip(label: Text('Dart')),
            Chip(label: Text('Widget')),
            Chip(label: Text('Layout')),
            Chip(label: Text('State')),
          ],
        ),
        const SizedBox(height: 16),
        const Text('3) Stack 层叠'),
        const SizedBox(height: 8),
        SizedBox(
          height: 130,
          child: Stack(
            children: <Widget>[
              Container(
                width: double.infinity,
                decoration: BoxDecoration(
                  color: Theme.of(context).colorScheme.primaryContainer,
                  borderRadius: BorderRadius.circular(12),
                ),
              ),
              const Positioned(left: 12, bottom: 12, child: Text('底层容器')),
              Positioned(
                top: _spacing * 1.2,
                right: _spacing * 1.2,
                child: const CircleAvatar(child: Icon(Icons.layers)),
              ),
            ],
          ),
        ),
        const SizedBox(height: 16),
        Text('间距：${_spacing.toStringAsFixed(0)}'),
        Slider(
          min: 4,
          max: 24,
          value: _spacing,
          onChanged: (double value) {
            setState(() {
              _spacing = value;
            });
          },
        ),
      ],
    ),
  );
 }
}

class _DemoColorBox extends StatelessWidget {
 const _DemoColorBox({required this.label});

 final String label;

 @override
 Widget build(BuildContext context) {
  return Container(
    height: 58,
    alignment: Alignment.center,
    decoration: BoxDecoration(
      color: Theme.of(context).colorScheme.secondaryContainer,
      borderRadius: BorderRadius.circular(10),
    ),
    child: Text(label),
  );
 }
}
```

4.10 lib/pages/button_interaction_page.dart

文件路径：`lib/pages/button_interaction_page.dart`

文件作用：演示多种按钮组件、启停控制和点击计数。

所属模块：按钮交互模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class ButtonInteractionPage extends StatefulWidget {
  const ButtonInteractionPage({super.key});

  @override
  State<ButtonInteractionPage> createState() => _ButtonInteractionPageState();
}

class _ButtonInteractionPageState extends State<ButtonInteractionPage> {
  int _clickCount = 0;
  bool _enabled = true;

  void _handleClick() {
    if (!_enabled) {
      return;
    }
    setState(() {
      _clickCount++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '按钮与点击交互',
      scenario: '提交、取消、跳转、收藏等操作都依赖按钮类组件，是交互入口最常见的一类组件。',
      tips: const <String>[
        '把 onPressed 改成 null，观察禁用态样式',
        '尝试修改按钮的 padding、shape、icon',
        '点击后让按钮文案跟随状态变化',
      ],
      keyCode: '''
ElevatedButton(
  onPressed: enabled ? onClick : null,
  child: const Text('主要按钮'),
)

setState(() {
  clickCount++;
});
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Text('当前点击次数：$_clickCount'),
          const SizedBox(height: 12),
          Wrap(
            spacing: 10,
            runSpacing: 10,
            children: <Widget>[
              ElevatedButton(
                onPressed: _enabled ? _handleClick : null,
                child: const Text('ElevatedButton'),
              ),
              OutlinedButton(
                onPressed: _enabled ? _handleClick : null,
                child: const Text('OutlinedButton'),
              ),
              TextButton(
                onPressed: _enabled ? _handleClick : null,
                child: const Text('TextButton'),
              ),
              IconButton(
                onPressed: _enabled ? _handleClick : null,
                icon: const Icon(Icons.thumb_up_alt_outlined),
              ),
              FilledButton.icon(
                onPressed: _enabled ? _handleClick : null,
                icon: const Icon(Icons.send),
                label: const Text('FilledButton'),
              ),
            ],
          ),
          const SizedBox(height: 12),
```

```
          SwitchListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('启用按钮'),
            value: _enabled,
            onChanged: (bool value) {
              setState(() {
                _enabled = value;
              });
            },
          ),
          Align(
            alignment: Alignment.centerRight,
            child: FloatingActionButton.small(
              onPressed: _enabled ? _handleClick : null,
              child: const Icon(Icons.add),
            ),
          ),
        ],
      ),
    );
  }
}
```

4.11 lib/pages/form_input_page.dart

文件路径：`lib/pages/form_input_page.dart`

文件作用：演示 TextField、下拉框、校验规则与表单提交。

所属模块：表单输入模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class FormInputPage extends StatefulWidget {
  const FormInputPage({super.key});

  @override
  State<FormInputPage> createState() => _FormInputPageState();
}

class _FormInputPageState extends State<FormInputPage> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _nameController = TextEditingController();

  String _city = '北京';
  bool _acceptProtocol = false;
  String _result = '尚未提交';

  @override
  void dispose() {
    _nameController.dispose();
    super.dispose();
  }

  void _submit() {
    final bool valid = _formKey.currentState?.validate() ?? false;
    if (!valid) {
      return;
    }
    if (!_acceptProtocol) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(const SnackBar(content: Text('请先勾选同意协议')));
      return;
    }
    setState(() {
      _result = '姓名：${_nameController.text}，城市：$_city';
    });
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(const SnackBar(content: Text('提交成功')));
  }

  @override
  Widget build(BuildContext context) {
```

```
    return DemoPageTemplate(
      title: '输入组件与表单',
      scenario: '登录注册、资料编辑、搜索筛选都离不开输入组件和表单校验。',
      tips: const <String>[
        '给 TextFormField 增加 maxLength 和 keyboardType',
        '修改 validator 的校验规则',
        '把 DropdownButtonFormField 的选项换成你自己的业务枚举',
      ],
      keyCode: '''
Form(
  key: formKey,
  child: TextFormField(
    controller: nameController,
    validator: (value) => value == null || value.isEmpty ? '请输入姓名' : null,
  ),
)
''',
      demoChild: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            TextFormField(
              controller: _nameController,
              decoration: const InputDecoration(
                labelText: '姓名',
                hintText: '请输入你的姓名',
                border: OutlineInputBorder(),
              ),
              validator: (String? value) {
                if (value == null || value.trim().isEmpty) {
                  return '请输入姓名';
                }
                if (value.trim().length < 2) {
                  return '姓名至少 2 个字符';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              value: _city,
              decoration: const InputDecoration(
                labelText: '城市',
                border: OutlineInputBorder(),
              ),
              items: const <DropdownMenuItem<String>>[
                DropdownMenuItem(value: '北京', child: Text('北京')),
                DropdownMenuItem(value: '上海', child: Text('上海')),
                DropdownMenuItem(value: '广州', child: Text('广州')),
              ],
              onChanged: (String? value) {
                if (value == null) {
                  return;
                }
                setState(() {
                  _city = value;
                });
              },
            ),
            const SizedBox(height: 12),
            CheckboxListTile(
              contentPadding: EdgeInsets.zero,
              value: _acceptProtocol,
              title: const Text('我已阅读并同意协议'),
              onChanged: (bool? value) {
                setState(() {
                  _acceptProtocol = value ?? false;
                });
              },
            ),
            const SizedBox(height: 12),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: _submit, child: const Text('提交')),
            ),
            const SizedBox(height: 12),
            Text('提交结果：$_result'),
          ],
        ),
```

```
      ),
    );
  }
}
```

4.12 lib/pages/feedback_page.dart

文件路径：`lib/pages/feedback_page.dart`

文件作用：演示 SnackBar、Dialog 与 BottomSheet 反馈场景。

所属模块：反馈提示模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class FeedbackPage extends StatelessWidget {
  const FeedbackPage({super.key});

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '弹窗与提示反馈',
      scenario: '用户执行操作后，需要及时反馈结果，比如成功提示、错误提醒或二次确认。',
      tips: const <String>[
        '把 SnackBar 的持续时间和行为按钮改掉',
        '尝试将 Dialog 换成 AlertDialog 或 SimpleDialog',
        '修改 BottomSheet 内容为你的业务菜单',
      ],
      keyCode: '''
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('保存成功')),
);

showDialog<void>(
  context: context,
  builder: (_) => const AlertDialog(title: Text('提示')),
);
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Wrap(
            spacing: 10,
            runSpacing: 10,
            children: <Widget>[
              FilledButton(
                onPressed: () {
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(
                      content: const Text('这是一个 SnackBar 提示'),
                      action: SnackBarAction(label: '撤销', onPressed: () {}),
                    ),
                  );
                },
                child: const Text('显示 SnackBar'),
              ),
              OutlinedButton(
                onPressed: () {
                  showDialog<void>(
                    context: context,
                    builder: (BuildContext context) {
                      return AlertDialog(
                        title: const Text('删除确认'),
                        content: const Text('确定要删除这条记录吗？'),
                        actions: <Widget>[
                          TextButton(
                            onPressed: () => Navigator.pop(context),
                            child: const Text('取消'),
                          ),
                          FilledButton(
                            onPressed: () => Navigator.pop(context),
                            child: const Text('确定'),
                          ),
                        ],
                      );
```

```
                    },
                  );
                },
                child: const Text('显示 Dialog'),
              ),
              ElevatedButton(
                onPressed: () {
                  showModalBottomSheet<void>(
                    context: context,
                    showDragHandle: true,
                    builder: (BuildContext context) {
                      return SafeArea(
                        child: Column(
                          mainAxisSize: MainAxisSize.min,
                          children: <Widget>[
                            ListTile(
                              leading: const Icon(Icons.edit_outlined),
                              title: const Text('编辑'),
                              onTap: () => Navigator.pop(context),
                            ),
                            ListTile(
                              leading: const Icon(Icons.delete_outline),
                              title: const Text('删除'),
                              onTap: () => Navigator.pop(context),
                            ),
                          ],
                        ),
                      );
                    },
                  );
                },
                child: const Text('显示 BottomSheet'),
              ),
            ],
          ),
        ],
      ),
    );
  }
}
```

4.13 lib/pages/navigation_page.dart

文件路径：`lib/pages/navigation_page.dart`

文件作用：演示 Navigator.push、pushNamed 与结果回传。

所属模块：页面导航模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';
import 'navigation/navigation_result.dart';
import 'navigation/route_detail_page.dart';

class NavigationPage extends StatefulWidget {
  const NavigationPage({super.key});

  @override
  State<NavigationPage> createState() => _NavigationPageState();
}

class _NavigationPageState extends State<NavigationPage> {
  String _resultText = '尚未收到返回结果';

  Future<void> _goByPush() async {
    final NavigationResult? result = await Navigator.push<NavigationResult>(
      context,
      MaterialPageRoute<NavigationResult>(
        builder: (BuildContext context) =>
            const RouteDetailPage(message: '来自 Navigator.push'),
      ),
    );
    if (result == null) {
      return;
    }
    setState(() {
```

```
      _resultText = '来源：${result.source}，时间：${result.timestamp.toLocal()}';
    });
  }

  Future<void> _goByNamedRoute() async {
    final NavigationResult? result =
        await Navigator.pushNamed<NavigationResult>(
          context,
          RouteDetailPage.routeName,
          arguments: '来自 pushNamed',
        );
    if (result == null) {
      return;
    }
    setState(() {
      _resultText = '来源：${result.source}，时间：${result.timestamp.toLocal()}';
    });
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '导航与页面切换',
      scenario: '多页面 App 会通过 Navigator 在页面间跳转，并经常需要传参与回传结果。',
      tips: const <String>[
        '把参数从字符串改成对象，体验复杂传参',
        '尝试 pushReplacement，观察返回行为差异',
        '给详情页增加更多返回数据字段',
      ],
      keyCode: '''
final result = await Navigator.pushNamed<ResultType>(
  context,
  '/route-detail',
  arguments: 'hello',
);
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Wrap(
            spacing: 10,
            runSpacing: 10,
            children: <Widget>[
              FilledButton(
                onPressed: _goByPush,
                child: const Text('使用 push 跳转'),
              ),
              OutlinedButton(
                onPressed: _goByNamedRoute,
                child: const Text('使用 pushNamed 跳转'),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Text('回传结果：$_resultText'),
        ],
      ),
    );
  }
}
```

4.14 lib/pages/navigation/route_detail_page.dart

文件路径：`lib/pages/navigation/route_detail_page.dart`

文件作用：接收页面参数并将处理结果返回上级页面。

所属模块：路由详情模块

```
import 'package:flutter/material.dart';
import 'navigation_result.dart';

class RouteDetailPage extends StatelessWidget {
  const RouteDetailPage({super.key, required this.message});

  static const String routeName = '/route-detail';
```

```
  final String message;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('详情页')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text('接收到的参数：$message'),
            const SizedBox(height: 16),
            FilledButton(
              onPressed: () {
                Navigator.pop(
                  context,
                  NavigationResult(source: message, timestamp: DateTime.now()),
                );
              },
              child: const Text('返回并携带结果'),
            ),
          ],
        ),
      ),
    );
  }
}
```

4.15 lib/pages/navigation/navigation_result.dart

文件路径：`lib/pages/navigation/navigation_result.dart`

文件作用：定义导航返回数据结构。

所属模块：导航结果模型

```
class NavigationResult {
  NavigationResult({required this.source, required this.timestamp});

  final String source;
  final DateTime timestamp;
}
```

4.16 lib/pages/state_lifting_page.dart

文件路径：`lib/pages/state_lifting_page.dart`

文件作用：演示父子组件状态同步、步长切换与回调传值。

所属模块：状态提升模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class StateLiftingPage extends StatefulWidget {
  const StateLiftingPage({super.key});

  @override
  State<StateLiftingPage> createState() => _StateLiftingPageState();
}

class _StateLiftingPageState extends State<StateLiftingPage> {
  int _count = 0;
  int _step = 1;

  void _add() {
    setState(() {
      _count += _step;
    });
  }
  void _minus() {
```

```
    setState(() {
      _count -= _step;
    });
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '状态更新与父子传值',
      scenario: '父组件保存状态，子组件通过回调通知父组件更新，是 Flutter 初学阶段最核心的状态模式。',
      tips: const <String>[
        '把 step 从下拉改成 Slider，体验不同交互方式',
        '在子组件中增加重置按钮，回调父组件清零',
        '尝试把 count 封装成独立模型对象再传递',
      ],
      keyCode: '''
ChildCounter(
  value: count,
  onAdd: handleAdd,
  onMinus: handleMinus,
)
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          _StepSelector(
            step: _step,
            onStepChanged: (int value) {
              setState(() {
                _step = value;
              });
            },
          ),
          const SizedBox(height: 12),
          ChildCounter(value: _count, onAdd: _add, onMinus: _minus),
        ],
      ),
    );
  }
}

class ChildCounter extends StatelessWidget {
  const ChildCounter({
    super.key,
    required this.value,
    required this.onAdd,
    required this.onMinus,
  });

  final int value;
  final VoidCallback onAdd;
  final VoidCallback onMinus;

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: EdgeInsets.zero,
      child: Padding(
        padding: const EdgeInsets.all(14),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text('当前值：$value', style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 10),
            Wrap(
              spacing: 10,
              children: <Widget>[
                FilledButton(onPressed: onAdd, child: const Text('+ 增加')),
                OutlinedButton(onPressed: onMinus, child: const Text('- 减少')),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

class _StepSelector extends StatelessWidget {
  const _StepSelector({required this.step, required this.onStepChanged});
```

```
  final int step;
  final ValueChanged<int> onStepChanged;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: <Widget>[
        const Text('步长：'),
        const SizedBox(width: 8),
        DropdownButton<int>(
          value: step,
          items: const <DropdownMenuItem<int>>[
            DropdownMenuItem(value: 1, child: Text('1')),
            DropdownMenuItem(value: 2, child: Text('2')),
            DropdownMenuItem(value: 5, child: Text('5')),
          ],
          onChanged: (int? value) {
            if (value == null) {
              return;
            }
            onStepChanged(value);
          },
        ),
      ],
    );
  }
}
```

4.17 lib/pages/theme_page.dart

文件路径：`lib/pages/theme_page.dart`

文件作用：演示浅色与深色主题切换及统一样式预览。

所属模块：主题样式模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class ThemeStudyPage extends StatelessWidget {
  const ThemeStudyPage({
    super.key,
    required this.currentMode,
    required this.onThemeModeChanged,
  });

  final ThemeMode currentMode;
  final ValueChanged<ThemeMode> onThemeModeChanged;

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '主题样式学习',
      scenario: '主题可以统一控制颜色、圆角、字体和亮暗模式，适合全局视觉风格管理。',
      tips: const <String>[
        '修改 seedColor，观察全局组件颜色联动',
        '给 CardTheme 或 InputDecorationTheme 增加统一样式',
        '尝试新增 ThemeMode.system，跟随系统亮暗主题',
      ],
      keyCode: '''
MaterialApp(
  theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue)),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
      brightness: Brightness.dark,
    ),
  ),
  themeMode: currentMode,
)
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          SegmentedButton<ThemeMode>(
            segments: const <ButtonSegment<ThemeMode>>[
```

```
                ButtonSegment<ThemeMode>(
                  value: ThemeMode.light,
                  label: Text('浅色'),
                  icon: Icon(Icons.light_mode),
                ),
                ButtonSegment<ThemeMode>(
                  value: ThemeMode.dark,
                  label: Text('深色'),
                  icon: Icon(Icons.dark_mode),
                ),
              ],
              selected: <ThemeMode>{currentMode},
              onSelectionChanged: (Set<ThemeMode> values) {
                onThemeModeChanged(values.first);
              },
            ),
            const SizedBox(height: 14),
            Card(
              child: Padding(
                padding: const EdgeInsets.all(12),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: <Widget>[
                    Text(
                      '主题预览卡片',
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                    const SizedBox(height: 8),
                    const Text('这段文字和按钮会自动跟随当前主题色与亮暗模式。'),
                    const SizedBox(height: 8),
                    FilledButton(onPressed: () {}, child: const Text('主题按钮')),
                  ],
                ),
              ),
            ),
          ],
        ),
      );
    }
}
```

4.18 lib/pages/async_list_page.dart

文件路径：`lib/pages/async_list_page.dart`

文件作用：演示 FutureBuilder、异常状态和异步列表刷新。

所属模块：异步加载模块

```
import 'package:flutter/material.dart';
import '../widgets/demo_page_template.dart';

class AsyncListPage extends StatefulWidget {
  const AsyncListPage({super.key});

  @override
  State<AsyncListPage> createState() => _AsyncListPageState();
}

class _AsyncListPageState extends State<AsyncListPage> {
  late Future<List<String>> _futureItems;
  bool _simulateError = false;

  @override
  void initState() {
    super.initState();
    _futureItems = _loadItems();
  }

  Future<List<String>> _loadItems() async {
    await Future<void>.delayed(const Duration(seconds: 2));
    if (_simulateError) {
      throw Exception('模拟网络异常');
    }
    return List<String>.generate(12, (int index) => '异步数据项 #${index + 1}');
  }
```

```
  void _reload() {
    setState(() {
      _futureItems = _loadItems();
    });
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '异步加载与列表展示',
      scenario: '请求接口、分页列表、下拉刷新都属于异步加载场景，FutureBuilder 是入门首选写法。',
      tips: const <String>[
        '把延迟从 2 秒改成 500ms，对比加载体验',
        '把 ListView.builder 改成 ListView.separated',
        '增加“空列表”分支，模拟无数据状态',
      ],
      keyCode:
          '''
FutureBuilder<List<String>>(
  future: futureItems,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) {
      return Text('加载失败: \${snapshot.error}');
    }
    return ListView.builder(...);
  },
)
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          SwitchListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('模拟接口错误'),
            value: _simulateError,
            onChanged: (bool value) {
              setState(() {
                _simulateError = value;
              });
            },
          ),
          const SizedBox(height: 8),
          FilledButton.icon(
            onPressed: _reload,
            icon: const Icon(Icons.refresh),
            label: const Text('重新加载'),
          ),
          const SizedBox(height: 12),
          FutureBuilder<List<String>>(
            future: _futureItems,
            builder:
                (BuildContext context, AsyncSnapshot<List<String>> snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Padding(
                  padding: EdgeInsets.symmetric(vertical: 20),
                  child: Center(child: CircularProgressIndicator()),
                );
              }
              if (snapshot.hasError) {
                return Text(
                  '加载失败：${snapshot.error}',
                  style: TextStyle(
                    color: Theme.of(context).colorScheme.error,
                  ),
                );
              }
              final List<String> items = snapshot.data ?? <String>[];
              if (items.isEmpty) {
                return const Text('暂无数据');
              }
              return ListView.separated(
                physics: const NeverScrollableScrollPhysics(),
                shrinkWrap: true,
                itemCount: items.length,
                separatorBuilder: (BuildContext context, int index) =>
                    const Divider(height: 1),
                itemBuilder: (BuildContext context, int index) {
```

```
                    return ListTile(
                      leading: const Icon(Icons.list_alt_outlined),
                      title: Text(items[index]),
                    );
                  },
                );
              },
            ),
          ],
        ),
      );
    }
  }
```

4.19 lib/pages/hardware_hub_page.dart

文件路径：`lib/pages/hardware_hub_page.dart`

文件作用：汇聚定位、传感器、相机、文件与蓝牙能力子页面。

所属模块：硬件入口模块

```
import 'package:flutter/material.dart';
import 'hardware/bluetooth_demo_page.dart';
import 'hardware/camera_demo_page.dart';
import 'hardware/file_read_demo_page.dart';
import 'hardware/gps_demo_page.dart';
import 'hardware/sensor_demo_page.dart';

class HardwareHubPage extends StatelessWidget {
  const HardwareHubPage({super.key});

  @override
  Widget build(BuildContext context) {
    final List<_HardwareItem> items = <_HardwareItem>[
      _HardwareItem(
        title: 'GPS 定位示例',
        subtitle: '获取权限 + 读取经纬度',
        icon: Icons.my_location,
        builder: (_) => const GpsDemoPage(),
      ),
      _HardwareItem(
        title: '陀螺仪/加速度计示例',
        subtitle: '实时读取传感器数据',
        icon: Icons.screen_rotation_alt,
        builder: (_) => const SensorDemoPage(),
      ),
      _HardwareItem(
        title: '相机示例',
        subtitle: '拍照并在页面中预览',
        icon: Icons.camera_alt_outlined,
        builder: (_) => const CameraDemoPage(),
      ),
      _HardwareItem(
        title: '文件读取示例',
        subtitle: '选择本地文件并读取文本',
        icon: Icons.insert_drive_file_outlined,
        builder: (_) => const FileReadDemoPage(),
      ),
      _HardwareItem(
        title: '蓝牙示例',
        subtitle: '打开蓝牙 + 扫描附近设备',
        icon: Icons.bluetooth_searching,
        builder: (_) => const BluetoothDemoPage(),
      ),
    ];

    return Scaffold(
      appBar: AppBar(title: const Text('硬件设备示例')),
      body: ListView.separated(
        padding: const EdgeInsets.all(16),
        itemCount: items.length,
        separatorBuilder: (_, __) => const SizedBox(height: 10),
        itemBuilder: (BuildContext context, int index) {
          final _HardwareItem item = items[index];
          return Card(
```

```
            child: ListTile(
              leading: CircleAvatar(child: Icon(item.icon)),
              title: Text(item.title),
              subtitle: Text(item.subtitle),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute<Widget>(builder: item.builder),
                );
              },
            ),
          );
        },
      ),
    );
  }
}

class _HardwareItem {
  _HardwareItem({
    required this.title,
    required this.subtitle,
    required this.icon,
    required this.builder,
  });

  final String title;
  final String subtitle;
  final IconData icon;
  final WidgetBuilder builder;
}
```

4.20 lib/pages/hardware/gps_demo_page.dart

文件路径：`lib/pages/hardware/gps_demo_page.dart`

文件作用：封装定位服务检查、权限申请与经纬度读取逻辑。

所属模块：定位能力模块

```
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
import '../../widgets/demo_page_template.dart';

class GpsDemoPage extends StatefulWidget {
  const GpsDemoPage({super.key});

  @override
  State<GpsDemoPage> createState() => _GpsDemoPageState();
}

class _GpsDemoPageState extends State<GpsDemoPage> {
  String _status = '点击按钮开始定位';

  Future<void> _getLocation() async {
    try {
      final bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
      if (!serviceEnabled) {
        setState(() {
          _status = '定位服务未开启，请先打开系统定位';
        });
        return;
      }

      LocationPermission permission = await Geolocator.checkPermission();
      if (permission == LocationPermission.denied) {
        permission = await Geolocator.requestPermission();
      }
      if (permission == LocationPermission.denied ||
          permission == LocationPermission.deniedForever) {
        setState(() {
          _status = '定位权限被拒绝';
        });
        return;
      }
```

```
      final Position position = await Geolocator.getCurrentPosition(
        desiredAccuracy: LocationAccuracy.high,
      );
      setState(() {
        _status =
            '纬度: ${position.latitude.toStringAsFixed(6)}\n经度: ${position.longitude.toStringAsFixed(6)}\n精度: ${position.accuracy.toStringAsFixed(1)} 米';
      });
    } catch (e) {
      setState(() {
        _status = '定位失败: $e';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: 'GPS 定位示例',
      scenario: '适用于外卖、地图、跑步记录、签到打卡等需要获取用户地理位置的场景。',
      tips: const <String>[
        '把精度从 high 改成 low，比较速度与精度',
        '尝试改成持续定位（位置流）',
        '加入经纬度到地址的逆地理编码',
      ],
      keyCode: '''
final position = await Geolocator.getCurrentPosition(
  desiredAccuracy: LocationAccuracy.high,
);
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          FilledButton.icon(
            onPressed: _getLocation,
            icon: const Icon(Icons.my_location),
            label: const Text('获取当前位置'),
          ),
          const SizedBox(height: 12),
          Text(_status),
        ],
      ),
    );
  }
}
```

4.21 lib/pages/hardware/sensor_demo_page.dart

文件路径：`lib/pages/hardware/sensor_demo_page.dart`

文件作用：订阅陀螺仪和加速度计数据流并实时更新界面。

所属模块：传感器能力模块

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:sensors_plus/sensors_plus.dart';
import '../../widgets/demo_page_template.dart';

class SensorDemoPage extends StatefulWidget {
  const SensorDemoPage({super.key});

  @override
  State<SensorDemoPage> createState() => _SensorDemoPageState();
}

class _SensorDemoPageState extends State<SensorDemoPage> {
  StreamSubscription<GyroscopeEvent>? _gyroSub;
  StreamSubscription<AccelerometerEvent>? _accSub;
  GyroscopeEvent? _gyro;
  AccelerometerEvent? _acc;
  bool _listening = false;

  void _startListen() {
    _gyroSub?.cancel();
    _accSub?.cancel();
    _gyroSub = gyroscopeEvents.listen((GyroscopeEvent event) {
```

```
      setState(() {
        _gyro = event;
      });
    });
    _accSub = accelerometerEvents.listen((AccelerometerEvent event) {
      setState(() {
        _acc = event;
      });
    });
    setState(() {
      _listening = true;
    });
  }

  Future<void> _stopListen() async {
    await _gyroSub?.cancel();
    await _accSub?.cancel();
    setState(() {
      _listening = false;
    });
  }

  @override
  void dispose() {
    _gyroSub?.cancel();
    _accSub?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '陀螺仪/加速度计示例',
      scenario: '适用于体感控制、步态识别、设备姿态判断、运动检测等场景。',
      tips: const <String>[
        '尝试自己加一个低通滤波，观察数据更平滑',
        '把数据保存到列表实现“最近 20 条”',
        '结合阈值实现“摇一摇”触发事件',
      ],
      keyCode: '''
gyroscopeEvents.listen((event) {
  setState(() => gyro = event);
});

accelerometerEvents.listen((event) {
  setState(() => acc = event);
});
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Wrap(
            spacing: 10,
            children: <Widget>[
              FilledButton(
                onPressed: _listening ? null : _startListen,
                child: const Text('开始监听'),
              ),
              OutlinedButton(
                onPressed: _listening ? _stopListen : null,
                child: const Text('停止监听'),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Text(
            '陀螺仪(rad/s):\n'
            'x=${_gyro?.x.toStringAsFixed(3) ?? '-'} '
            'y=${_gyro?.y.toStringAsFixed(3) ?? '-'} '
            'z=${_gyro?.z.toStringAsFixed(3) ?? '-'}',
          ),
          const SizedBox(height: 8),
          Text(
            '加速度(m/s²):\n'
            'x=${_acc?.x.toStringAsFixed(3) ?? '-'} '
            'y=${_acc?.y.toStringAsFixed(3) ?? '-'} '
            'z=${_acc?.z.toStringAsFixed(3) ?? '-'}',
          ),
        ],
      ),
```

```
    );
  }
}
```

4.22 lib/pages/hardware/camera_demo_page.dart

文件路径：`lib/pages/hardware/camera_demo_page.dart`

文件作用：通过 image_picker 调起相机并回显拍照结果。

所属模块：相机能力模块

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import '../../widgets/demo_page_template.dart';

class CameraDemoPage extends StatefulWidget {
  const CameraDemoPage({super.key});

  @override
  State<CameraDemoPage> createState() => _CameraDemoPageState();
}

class _CameraDemoPageState extends State<CameraDemoPage> {
  final ImagePicker _picker = ImagePicker();
  XFile? _photo;
  String _status = '还没有拍照';

  Future<void> _takePhoto() async {
    try {
      final XFile? file = await _picker.pickImage(
        source: ImageSource.camera,
        imageQuality: 85,
      );
      if (file == null) {
        setState(() {
          _status = '用户取消了拍照';
        });
        return;
      }
      setState(() {
        _photo = file;
        _status = '拍照成功：${file.path}';
      });
    } catch (e) {
      setState(() {
        _status = '拍照失败: $e';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '相机示例',
      scenario: '适用于头像上传、工单拍照、扫码前置拍摄、OCR 拍照上传等场景。',
      tips: const <String>[
        '修改 imageQuality，比较清晰度和体积',
        '改成 ImageSource.gallery 做相册选择',
        '拍照后增加裁剪/压缩步骤',
      ],
      keyCode: '''
final photo = await ImagePicker().pickImage(
  source: ImageSource.camera,
  imageQuality: 85,
);
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          FilledButton.icon(
            onPressed: _takePhoto,
            icon: const Icon(Icons.camera_alt_outlined),
            label: const Text('打开相机拍照'),
          ),
```

```
          const SizedBox(height: 12),
          Text(_status),
          const SizedBox(height: 12),
          if (_photo != null)
            ClipRRect(
              borderRadius: BorderRadius.circular(12),
              child: Image.file(
                File(_photo!.path),
                height: 220,
                fit: BoxFit.cover,
              ),
            ),
        ],
      ),
    );
  }
}
```

4.23 lib/pages/hardware/file_read_demo_page.dart

文件路径：`lib/pages/hardware/file_read_demo_page.dart`

文件作用：通过 file_picker 选择本地文件并预览文本内容。

所属模块：文件读取模块

```
import 'dart:io';
import 'package:file_picker/file_picker.dart';
import 'package:flutter/material.dart';
import '../../widgets/demo_page_template.dart';

class FileReadDemoPage extends StatefulWidget {
  const FileReadDemoPage({super.key});

  @override
  State<FileReadDemoPage> createState() => _FileReadDemoPageState();
}

class _FileReadDemoPageState extends State<FileReadDemoPage> {
  String _status = '请选择一个 txt/json 文件';
  String _contentPreview = '';

  Future<void> _pickAndReadFile() async {
    try {
      final FilePickerResult? result = await FilePicker.platform.pickFiles(
        type: FileType.custom,
        allowedExtensions: <String>['txt', 'json', 'md'],
      );
      if (result == null || result.files.single.path == null) {
        setState(() {
          _status = '用户取消选择';
        });
        return;
      }
      final String filePath = result.files.single.path!;
      final String content = await File(filePath).readAsString();
      setState(() {
        _status = '读取成功: $filePath';
        _contentPreview = content.length > 400
            ? '${content.substring(0, 400)}...'
            : content;
      });
    } catch (e) {
      setState(() {
        _status = '读取失败: $e';
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '文件读取示例',
      scenario: '适用于导入配置、读取日志、选择本地文档、离线数据分析等场景。',
      tips: const <String>[
        '把允许后缀扩展为 csv、xml 等',
```

```
        '把 readAsString 改成 readAsBytes 读取二进制',
        '加入文件大小限制提示',
      ],
      keyCode: '''
final result = await FilePicker.platform.pickFiles();
final content = await File(result.files.single.path!).readAsString();
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          FilledButton.icon(
            onPressed: _pickAndReadFile,
            icon: const Icon(Icons.folder_open),
            label: const Text('选择并读取文件'),
          ),
          const SizedBox(height: 12),
          Text(_status),
          const SizedBox(height: 12),
          if (_contentPreview.isNotEmpty)
            Container(
              width: double.infinity,
              padding: const EdgeInsets.all(12),
              decoration: BoxDecoration(
                color: Theme.of(context).colorScheme.surfaceContainerHighest,
                borderRadius: BorderRadius.circular(10),
              ),
              child: SelectableText(_contentPreview),
            ),
        ],
      ),
    );
  }
}
```

4.24 lib/pages/hardware/bluetooth_demo_page.dart

文件路径：`lib/pages/hardware/bluetooth_demo_page.dart`

文件作用：控制蓝牙扫描并展示附近设备列表。

所属模块：蓝牙能力模块

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import '../../widgets/demo_page_template.dart';

class BluetoothDemoPage extends StatefulWidget {
  const BluetoothDemoPage({super.key});

  @override
  State<BluetoothDemoPage> createState() => _BluetoothDemoPageState();
}

class _BluetoothDemoPageState extends State<BluetoothDemoPage> {
  final List<ScanResult> _results = <ScanResult>[];
  StreamSubscription<List<ScanResult>>? _scanSub;
  bool _scanning = false;
  String _status = '尚未扫描';

  Future<void> _scan() async {
    try {
      _results.clear();
      setState(() {
        _scanning = true;
        _status = '正在扫描 6 秒...';
      });

      await FlutterBluePlus.turnOn();
      _scanSub?.cancel();
      _scanSub = FlutterBluePlus.scanResults.listen((List<ScanResult> data) {
        setState(() {
          _results
            ..clear()
            ..addAll(data);
        });
```

```
      });
      await FlutterBluePlus.startScan(timeout: const Duration(seconds: 6));
      setState(() {
        _scanning = false;
        _status = '扫描完成，找到 ${_results.length} 个设备';
      });
    } catch (e) {
      setState(() {
        _scanning = false;
        _status = '扫描失败: $e';
      });
    }
  }

  @override
  void dispose() {
    _scanSub?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return DemoPageTemplate(
      title: '蓝牙示例',
      scenario: '适用于连接手环、温度计、蓝牙打印机、物联网设备等近场通信场景。',
      tips: const <String>[
        '把扫描时间从 6 秒改成 10 秒',
        '按设备名称过滤列表',
        '后续可继续加“连接设备并读写特征值”',
      ],
      keyCode: '''
await FlutterBluePlus.startScan(timeout: const Duration(seconds: 6));
FlutterBluePlus.scanResults.listen((results) {
  setState(() => devices = results);
});
''',
      demoChild: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          FilledButton.icon(
            onPressed: _scanning ? null : _scan,
            icon: const Icon(Icons.bluetooth_searching),
            label: Text(_scanning ? '扫描中...' : '开始蓝牙扫描'),
          ),
          const SizedBox(height: 12),
          Text(_status),
          const SizedBox(height: 12),
          if (_results.isNotEmpty)
            ListView.separated(
              shrinkWrap: true,
              physics: const NeverScrollableScrollPhysics(),
              itemCount: _results.length,
              separatorBuilder: (_, __) => const Divider(height: 1),
              itemBuilder: (BuildContext context, int index) {
                final ScanResult result = _results[index];
                final String name = result.device.platformName.isEmpty
                    ? '未命名设备'
                    : result.device.platformName;
                return ListTile(
                  title: Text(name),
                  subtitle: Text(result.device.remoteId.str),
                  trailing: Text('RSSI ${result.rssi}'),
                );
              },
            ),
        ],
      ),
    );
  }
}
```

4.25 lib/widgets/demo_page_template.dart

文件路径：`lib/widgets/demo_page_template.dart`

文件作用：统一演示页面的场景说明、提示、代码区和效果区布局。

所属模块：页面模板组件

```
import 'package:flutter/material.dart';
import 'code_block.dart';
import 'section_card.dart';

class DemoPageTemplate extends StatelessWidget {
  const DemoPageTemplate({
    super.key,
    required this.title,
    required this.scenario,
    required this.tips,
    required this.keyCode,
    required this.demoChild,
  });

  final String title;
  final String scenario;
  final List<String> tips;
  final String keyCode;
  final Widget demoChild;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(title)),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            SectionCard(title: '这个组件的使用场景', child: Text(scenario)),
            SectionCard(
              title: '你可以尝试修改这些参数',
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: tips
                    .map(
                      (String tip) => Padding(
                        padding: const EdgeInsets.only(bottom: 6),
                        child: Text('• $tip'),
                      ),
                    )
                    .toList(),
              ),
            ),
            SectionCard(
              title: '关键代码展示',
              child: CodeBlock(code: keyCode),
            ),
            SectionCard(title: '运行效果', child: demoChild),
          ],
        ),
      ),
    );
  }
}
```

4.26 lib/widgets/section_card.dart

文件路径：`lib/widgets/section_card.dart`

文件作用：统一承载模块页面中的分区说明内容。

所属模块：通用卡片组件

```
import 'package:flutter/material.dart';

class SectionCard extends StatelessWidget {
  const SectionCard({super.key, required this.title, required this.child});

  final String title;
  final Widget child;

  @override
  Widget build(BuildContext context) {
```

```
    return Card(
      margin: const EdgeInsets.only(bottom: 12),
      child: Padding(
        padding: const EdgeInsets.all(14),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(
              title,
              style: Theme.of(
                context,
              ).textTheme.titleMedium?.copyWith(fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 10),
            child,
          ],
        ),
      ),
    );
  }
}
```

4.27 lib/widgets/code_block.dart

文件路径：`lib/widgets/code_block.dart`

文件作用：负责高可读性展示示例代码片段。

所属模块：代码展示组件

```
import 'package:flutter/material.dart';

class CodeBlock extends StatelessWidget {
  const CodeBlock({super.key, required this.code});

  final String code;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      decoration: BoxDecoration(
        color: Theme.of(context).colorScheme.surfaceContainerHighest,
        borderRadius: BorderRadius.circular(10),
      ),
      padding: const EdgeInsets.all(12),
      child: SelectableText(
        code.trim(),
        style: const TextStyle(
          fontFamily: 'monospace',
          fontSize: 13,
          height: 1.4,
        ),
      ),
    );
  }
}
```

4.28 lib/widgets/network_image_with_fallback.dart

文件路径：`lib/widgets/network_image_with_fallback.dart`

文件作用：加载网络图片并在异常时提供本地图标兜底。

所属模块：图片兜底组件

```
import 'package:flutter/material.dart';

class NetworkImageWithFallback extends StatelessWidget {
  const NetworkImageWithFallback({
    super.key,
    required this.url,
```

```
    this.height = 140,
  });

  final String url;
  final double height;

  @override
  Widget build(BuildContext context) {
    return ClipRRect(
      borderRadius: BorderRadius.circular(12),
      child: Image.network(
        url,
        height: height,
        width: double.infinity,
        fit: BoxFit.cover,
        loadingBuilder:
            (BuildContext context, Widget child, ImageChunkEvent? progress) {
          if (progress == null) {
            return child;
          }
          return SizedBox(
            height: height,
            child: const Center(child: CircularProgressIndicator()),
          );
        },
        errorBuilder:
            (BuildContext context, Object error, StackTrace? stackTrace) {
          return Container(
            height: height,
            color: Theme.of(context).colorScheme.surfaceContainerHighest,
            alignment: Alignment.center,
            child: const Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: <Widget>[
                Icon(Icons.broken_image_outlined, size: 36),
                SizedBox(height: 6),
                Text('图片加载失败，已显示兜底视图'),
              ],
            ),
          );
        },
      ),
    );
  }
}
```